TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN — MODALIDAD A DISTANCIA

PROGRAMACIÓN IV

SEGUNDA EVALUACIÓN PARCIAL —Parcial de Desarrollo

1. Objetivo

Evaluar de forma sumativa los contenidos de las **Unidades 4, 5 y 6**: construcción de una API REST con FastAPI y Pydantic, persistencia con SQLModel sobre PostgreSQL, modelado de relaciones 1:N y N:N, e interfaz React + TypeScript con estado local y estilos Tailwind CSS. El dominio de trabajo es **Catálogo de Productos**. Se evalúa la implementación completa de extremo a extremo: contrato HTTP, capa de persistencia, integridad del modelo relacional y consumo de la API desde el frontend.

2. Contexto del dominio

Food Store es un negocio de comidas que necesita informatizar su catálogo. El catálogo se administra desde una interfaz web y está formado por dos cosas: las **categorías**, que organizan la oferta en rubros (por ejemplo «Pastas», «Postres», «Bebidas»), y los **productos**, que se dan de alta con su precio, la unidad en la que se venden, su stock y las categorías en las que se clasifican.

La regla central del dominio es que **cada producto lleva su propio stock**, expresado en su unidad de venta: si se vende por kilogramo, el stock se registra en kilogramos; si se vende por pieza, en piezas. Que un producto tenga existencias se deriva de ese stock. Además, el administrador puede pausar la venta de un producto (`disponible`) sin darlo de baja, y darlo de baja (`activo`) sin perder su historial, porque en Food Store nada se elimina físicamente.

3. Modelo de datos de partida

El modelo se entrega completo: entidades, campos, restricciones y relaciones.

3.1 Entidades

Entidad	Campos	Restricciones
UnidadMedida	id, nombre, simbolo, tipo	id PK. nombre y simbolo únicos. tipo es un `Enum` con los valores peso, volumen y contable. Catálogo de solo lectura con seed inicial (Requerimiento 1).
Categoria	id, nombre, descripcion, activo, anulado_at	id PK. nombre obligatorio (mín. 3) y único entre las categorías activas. descripcion opcional. activo BOOLEAN NOT NULL default true. anulado_at TIMESTAMP nullable.
Producto	id, unidad_venta_id, nombre, descripcion, precio_base, stock_cantidad, disponible, activo, anulado_at	id PK. unidad_venta_id FK → UnidadMedida, NOT NULL. nombre obligatorio (mín. 3) y único entre los activos. precio_base numérico ≥ 0 (dos decimales). stock_cantidad numérico ≥ 0 (tres decimales), default 0, expresado en la unidad de venta. disponible BOOLEAN NOT NULL default true: pausa manual de la venta, independiente de activo. activo BOOLEAN NOT NULL default true. anulado_at TIMESTAMP nullable.

Entidad	Campos	Restricciones
ProductoCategoria	producto_id, categoria_id, es_principal	PK compuesta (producto_id, categoria_id). es_principal BOOLEAN NOT NULL default false; a lo sumo una fila con es_principal = true por producto.

3.2 Diagrama entidad-relación (Mermaid)

```
erDiagram
    UNIDAD_MEDIDA {
        int id PK
        string nombre
        string simbolo
        string tipo
    }
    CATEGORIA {
        int id PK
        string nombre
        string descripcion
        bool activo
        datetime anulado_at
    }
    PRODUCTO {
        int id PK
        int unidad_venta_id FK
        string nombre
        string descripcion
        decimal precio_base
        decimal stock_cantidad
        bool disponible
        bool activo
        datetime anulado_at
    }
    PRODUCTO_CATEGORIA {
        int producto_id PK,FK
        int categoria_id PK,FK
        bool es_principal
    }

    UNIDAD_MEDIDA  ||--o{ PRODUCTO             : "unidad de venta"
    PRODUCTO       ||--o{ PRODUCTO_CATEGORIA   : "clasifica"
    CATEGORIA      ||--o{ PRODUCTO_CATEGORIA   : "agrupa"
```

3.3 Estados de un producto: activo, disponible y hay_stock

Un producto del catálogo tiene tres indicadores que responden preguntas distintas y que el backend nunca mezcla. Dos se persisten y uno se calcula:

Indicador	Pregunta que responde	Cómo se modifica	Efecto en la API
activo	¿El registro sigue vigente en el catálogo?	`DELETE /api/v1/productos/{id}` realiza la baja lógica: activo = false y anulado_at con la marca de tiempo. No hay borrado físico.	activo = false lo saca del listado por defecto (se lo puede consultar con el filtro activo=false y por su id) y rechaza cualquier modificación posterior con 409.
disponible	¿Se puede vender ahora mismo?	Campo del cuerpo de PUT `/api/v1/productos/{id}`. Persistido. Independiente de activo.	El producto sigue apareciendo en el catálogo; el frontend lo muestra con el indicador de venta desactivado.

Indicador	Pregunta que responde	Cómo se modifica	Efecto en la API
hay_stock	¿Hay existencias para vender?	No se modifica directamente: se deriva de stock_cantidad en cada lectura.	Campo calculado de ProductoPublic: true cuando stock_cantidad > 0.

Combinaciones válidas: un producto puede estar activo = true y disponible = false (pausado); activo = true, disponible = true y hay_stock = false (sin existencias); o activo = false con cualquier valor de los otros dos (dado de baja, conserva su última configuración). Ninguna operación sobre un indicador altera a los otros.

3.4 Reglas de negocio

El backend debe hacer cumplir las siguientes reglas. Cada una es verificable desde los archivos `.http` de la entrega.

ID	Regla	Verificación
RN-01	nombre es obligatorio (mín. 3 caracteres) en Producto y Categoria; precio_base ≥ 0; stock_cantidad ≥ 0. Se valida en el schema Pydantic con `Field`.	422
RN-02	El nombre es único entre los productos activos y entre las categorías activas. Un registro anulado libera su nombre.	409
RN-03	No se puede consultar, actualizar ni anular una entidad cuyo id no exista; tampoco referenciar por FK un id inexistente (unidad o categoría).	404
RN-04	Categoria y Producto se dan de baja con `DELETE` mediante anulación lógica (activo = false y anulado_at con la marca de tiempo). No existe borrado físico. Anular un registro ya anulado se rechaza.	409
RN-05	Un registro anulado no admite modificación: PUT sobre un producto o una categoría anulados se rechaza.	409
RN-06	Anular una categoría no anula los productos clasificados en ella. Cada registro conserva su propio estado.	REST Client
RN-07	Un producto no puede clasificarse dos veces en la misma categoría.	409
RN-08	No se puede clasificar un producto en una categoría anulada ni clasificar un producto anulado.	409
RN-09	Un producto tiene como máximo una categoría principal: al clasificarlo en una nueva categoría con es_principal = true, la anterior deja de serlo en la misma operación.	201 y luego GET con el vínculo anterior en false
RN-10	Si la unidad de venta del producto es de tipo contable, stock_cantidad debe ser un número entero. Se valida al crear y al actualizar, también cuando cambia unidad_venta_id.	422
RN-11	hay_stock es verdadero cuando stock_cantidad > 0. Se calcula en cada lectura y no se persiste en Producto.	REST Client
RN-12	disponible, activo y stock_cantidad son independientes: ninguna operación sobre uno modifica los otros.	REST Client
RN-13	Toda condición de error se informa con `HTTPException` en formato `{"detail": "..."}`. Las validaciones se realizan antes de persistir: una petición rechazada no modifica la base.	404 / 409 / 422

4. Requerimientos obligatorios

Los cuatro requerimientos suman 100 puntos. El puntaje de cada uno figura en su título y se detalla en la grilla de la sección 8.

Requerimiento 1 — Backend: estructura, persistencia y modelo de datos (15 pts)

- Proyecto FastAPI con arquitectura modular por feature: `app/modules/categorias/` y `app/modules/productos/`, cada uno con `router.py`, `model.py`, `schema.py` y `service.py`; y `app/modules/unidades_medida/` con `model.py` y `router.py` (catálogo de solo lectura).
- Persistencia con SQLModel sobre PostgreSQL. El `engine` y la dependencia de sesión (`Session`) se definen en `app/core/database.py`; los routers obtienen la sesión por inyección de dependencias y no la construyen.
- Las cuatro entidades de la sección 3.1 implementadas como modelos SQLModel (`table=True`) con sus claves primarias, claves foráneas, PK compuestas y restricciones. `UnidadMedida.tipo` se declara con un `Enum`.
- Los routers no contienen lógica de negocio ni acceso directo a la base: delegan en `service.py`.
- Seed inicial de UnidadMedida en `app/core/seed.py` con las seis unidades siguientes: kilogramo (kg) y gramo (g) de tipo peso; litro (L) y mililitro (mL) de tipo volumen; pieza (u) y docena (doc) de tipo contable. El seed solo inserta si la tabla está vacía: ejecutar la aplicación dos veces no duplica filas.
- `GET /api/v1/unidades-medida` (200) devuelve el catálogo de unidades; es el origen del selector de unidad de venta del frontend.
- CORS habilitado con `CORSMiddleware` para el origen del frontend.
- Swagger operativo en `/docs` con todos los endpoints documentados.
- Entorno de trabajo: entorno virtual `.venv` y `requirements.txt` con las dependencias del proyecto.

Requerimiento 2 — Módulo Productos: CRUD, stock, validaciones y consultas (30 pts)

Implementar el CRUD completo de Producto con prefijo `/api/v1`:

Método	Endpoint	Código	Descripción
POST	/api/v1/productos	201	Crear un producto (body: nombre, descripcion, precio_base, unidad_venta_id, stock_cantidad opcional default 0, disponible)
GET	/api/v1/productos	200	Listar con paginación y filtros
GET	/api/v1/productos/{id}	200	Detalle con su unidad de venta, sus categorías, stock_cantidad y hay_stock
PUT	/api/v1/productos/{id}	200	Actualizar los campos propios del producto: nombre, descripcion, precio_base, unidad_venta_id, stock_cantidad y disponible
DELETE	/api/v1/productos/{id}	204	Anular el producto (baja lógica: activo = false y anulado_at)

Parámetros de consulta de GET /api/v1/productos:

Parámetro	Tipo	Default	Validación / efecto
skip	int	0	ge=0

Parámetro	Tipo	Default	Validación / efecto
limit	int	10	ge=1, le=50
activo	bool | None	None	Omitido o true devuelve solo los activos; false devuelve solo los anulados
disponible	bool | None	None	Filtra por el indicador disponible; omitido no filtra
categoria_id	int | None	None	Devuelve solo los productos clasificados en esa categoría (vía ProductoCategoria), sin duplicar filas
search	str | None	None	Coincidencia parcial sobre el nombre, sin distinguir mayúsculas

- Schemas Pydantic separados para entrada y salida: `ProductoBase`, `ProductoCreate`, `ProductoUpdate` (mismos campos que Create: PUT reemplaza los campos propios) y `ProductoPublic`. El modelo de tabla nunca se expone directamente como response: todos los endpoints declaran `response_model`.
- Validaciones declaradas en el schema con `Annotated` + `Field`: nombre con `min_length=3`, precio_base y stock_cantidad con `ge=0`, unidad_venta_id obligatorio. Los parámetros del listado se declaran con `Query` y el id con `Path`.
- El listado devuelve `list[ProductoPublic]`. `ProductoPublic` incluye la unidad de venta anidada, las categorías, stock_cantidad y hay_stock como campo calculado (RN-11).
- `DELETE /api/v1/productos/{id}` fija activo = false y registra anulado_at; nunca borra la fila. Sobre un producto ya anulado devuelve 409.
- El servicio valida RN-10 antes de persistir: un stock_cantidad con decimales sobre una unidad de venta contable devuelve 422, tanto en POST como en PUT. hay_stock se calcula en el servicio y no se persiste.
- Errores devueltos con `HTTPException` en formato `{"detail": "..."}`, con los códigos de la sección 3.4.
- Archivo `tests/productos.http` que demuestre los cinco endpoints, los cuatro filtros del listado, los escenarios de error 404, 409 y 422 (incluidos stock negativo y stock con decimales sobre unidad contable) y la lectura de hay_stock antes y después de llevar el stock a 0.

Requerimiento 3 — Relaciones 1:N y N:N (25 pts)

Relación 1:N — UnidadMedida y Producto

- La clave foránea unidad_venta_id reside en Producto y es obligatoria: todo producto se vende en una unidad.
- Ambos lados declaran `Relationship` con `back_populates`.
- `ProductoPublic` incluye la unidad anidada (al menos nombre y simbolo), no solo el id. Un unidad_venta_id inexistente devuelve 404.

Módulo Categorías

El módulo `categorias/` expone el CRUD de la tabla siguiente, con los mismos criterios de schemas, `response_model` y errores que Productos.

Método	Endpoint	Código	Descripción
POST	/api/v1/categorias	201	Crear una categoría (body: nombre, descripcion opcional)
GET	/api/v1/categorias	200	Listar con los filtros activo y search

Método	Endpoint	Código	Descripción
GET	/api/v1/categorias/{id}	200	Detalle de la categoría
PUT	/api/v1/categorias/{id}	200	Actualizar nombre y descripcion
DELETE	/api/v1/categorias/{id}	204	Anular la categoría (baja lógica)

- Se cumplen RN-02 (nombre único entre activas) y RN-06 (anular una categoría no afecta a sus productos).

Relación N:N — Producto y Categoría

- La relación se resuelve con el modelo intermedio `ProductoCategoria` (link model), con clave primaria compuesta y el atributo propio es_principal. Ambos lados se navegan con `Relationship` y `link_model`.

Método	Endpoint	Código	Descripción
POST	/api/v1/productos/{id}/categorias	201	Clasificar el producto en una categoría (body: categoria_id, es_principal)
GET	/api/v1/productos/{id}/categorias	200	Listar las categorías del producto indicando cuál es la principal

- Se cumplen RN-07, RN-08 y RN-09. Clasificar en una categoría anulada, o un producto anulado, devuelve 409.
- Archivo `tests/relaciones.http` que demuestre: alta de categorías, nombre repetido (409), anulación de una categoría y verificación de que sus productos siguen activos, clasificación de un producto en dos categorías con cambio de principal, intento de clasificar dos veces en la misma categoría (409) y filtro `categoria_id` del listado de productos.

Requerimiento 4 — Frontend React + TypeScript + Tailwind (30 pts)

Implementar la administración del catálogo de productos en una aplicación React + TypeScript creada con Vite y estilada con Tailwind CSS 4. Solo el CRUD de Productos se opera desde la interfaz; las categorías y la clasificación de productos se demuestran con REST Client.

Tipado y estructura

- Interfaces en `src/types/` para `Producto`, `UnidadMedida` y `Categoria`, con los mismos nombres de campo que devuelve la API (incluidos stock_cantidad y hay_stock). Compilación con `strict: true` y sin any.
- Componentes funcionales en `src/components/` con props tipadas mediante interfaces: barra de navegación; listado que recibe `Producto[]` por props y lo renderiza con `.map()`; tarjeta o fila con el precio y el stock acompañados del símbolo de la unidad («$ 12,50 / kg», «stock: 3,5 kg»), los indicadores de disponible y hay_stock, y los botones Editar, Disponibilidad y Anular; y modal de alta/edición con el campo de stock y el selector de unidad de venta cargado desde `GET /api/v1/unidades-medida`.
- Estilos con Tailwind CSS 4 con enfoque utility-first: contenedor, tarjetas, tipografía, colores, inputs, labels y botones con sus estados (hover, disabled).

Estado y consumo de la API

- En `App.tsx`: `useState` para la lista de productos, la lista de unidades, la apertura del modal, el producto seleccionado y los estados de carga y error; `useEffect` para cargar el catálogo y las unidades al montar.
- El estado del formulario del modal se maneja con `useState` (un objeto con los campos del formulario o un estado por campo) y se inicializa vacío al crear o con los datos del producto al editar.

- Alta (`POST`), edición (`PUT`), cambio de disponibilidad (`PUT` con disponible invertido) y anulación (`DELETE`) resueltos con `fetch` nativo contra la API, refrescando el estado tras cada operación.
- Levantamiento de estado: el estado del catálogo vive en `App.tsx`; los componentes hijos lo reciben por props y comunican las acciones mediante funciones recibidas por props. Ningún componente hijo consulta la API por su cuenta.
- El modal se abre para crear y para editar, y se cierra al guardar o al cancelar. Un 409 de la API (nombre repetido) se muestra dentro del modal sin cerrarlo.
- La interfaz contempla los estados de carga, de error y de lista vacía.

5. Estructura del proyecto

La entrega contiene dos proyectos. Ambos deben respetar la organización indicada.

```
foodstore-backend/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── core/
│   │   ├── __init__.py
│   │   ├── database.py
│   │   └── seed.py
│   └── modules/
│       ├── unidades_medida/    # __init__ · model · router
│       ├── categorias/         # __init__ · router · model · schema · service
│       └── productos/          # ídem · más ProductoCategoria
├── tests/                      # productos.http · relaciones.http
├── requirements.txt
└── README.md
```

```
foodstore-frontend/
├── src/
│   ├── components/
│   ├── types/
│   ├── App.tsx
│   ├── main.tsx
│   └── index.css
├── index.html
├── package.json
├── tsconfig.json
└── README.md
```

6. Condiciones de entrega

6.1 Requisitos técnicos

- Backend: Python 3.12 o superior, FastAPI, SQLModel, Pydantic v2 y PostgreSQL 15 o superior.
- No se admite almacenamiento en memoria: la persistencia es en PostgreSQL. No se permiten ORMs alternativos ni SQLAlchemy declarado por fuera de SQLModel.
- Todos los endpoints usan el prefijo `/api/v1`, declaran `response_model` y devuelven los errores en formato `{"detail": "..."}`.
- La baja de todo recurso es lógica: `DELETE` marca activo = false y anulado_at; ninguna fila se borra físicamente.
- La cadena de conexión a PostgreSQL se define en `app/core/database.py`. El README indica cómo configurarla.
- Frontend: React + TypeScript sobre Vite, con Tailwind CSS 4. Se usa `fetch` nativo; no se permiten Axios, TanStack Query, Zustand ni librerías de formularios o de componentes.

- El manejo de estado se resuelve con `useState` y `useEffect`.

6.2 Código

- Un único archivo `Apellido_Nombre_P2.zip` con las dos carpetas del proyecto.
- Antes de comprimir, eliminar `__pycache__/`, `.venv/`, `node_modules/` y `dist/`.
- Cada proyecto incluye su `README.md` con los pasos de instalación y ejecución.
- Al descomprimir en una máquina limpia debe funcionar: `pip install -r requirements.txt` y `fastapi dev app/main.py` en el backend; `npm install` y `npm run dev` en el frontend.
- Los archivos `.http` de prueba se entregan dentro de `tests/`.

7. Historias de usuario

Las historias siguen el formato «Como… quiero… para…». Los criterios de aceptación son la condición verificable de que la historia fue implementada.

HU-P2-01: Administrar los productos del catálogo

Como administrador de Food Store, **quiero** dar de alta, consultar, actualizar, pausar y anular productos y registrar su stock, **para** mantener el catálogo del local al día sin perder su historial.

Criterios de aceptación

- `POST /api/v1/productos` crea el producto a partir de un cuerpo validado con Pydantic y responde 201 con el producto creado, incluidos su id, su unidad de venta anidada y hay_stock.
- Un cuerpo inválido (nombre de menos de 3 caracteres, precio o stock negativos, o sin unidad de venta) se rechaza con 422 sin alterar la base.
- Un nombre repetido entre los productos activos se rechaza con 409, tanto al crear como al actualizar.
- `GET /api/v1/productos` devuelve la lista paginada de productos activos y admite los filtros activo, disponible, categoria_id y search.
- `PUT /api/v1/productos/{id}` actualiza los campos propios del producto y responde 200 con el producto actualizado; un id inexistente devuelve 404 y un producto anulado devuelve 409.
- `PUT /api/v1/productos/{id}` con disponible = false lo deja en el catálogo con la venta pausada y no modifica activo.
- Con una unidad de venta contable, un stock_cantidad con decimales se rechaza con 422. hay_stock es verdadero solo cuando stock_cantidad > 0 y se actualiza al modificar el stock.
- `DELETE /api/v1/productos/{id}` fija activo = false y anulado_at, responde 204 y lo saca del listado por defecto, sin modificar disponible. Repetir la anulación devuelve 409.
- Cualquier operación sobre un id inexistente devuelve 404.

HU-P2-02: Organizar el catálogo en categorías y clasificar productos

Como administrador de Food Store, **quiero** organizar el catálogo en categorías y asociar cada producto a una o más de ellas, **para** que el cliente encuentre los productos por rubro.

Criterios de aceptación

- `POST /api/v1/categorias` crea la categoría y responde 201; un nombre de menos de 3 caracteres devuelve 422 y un nombre repetido entre las activas devuelve 409.
- `PUT /api/v1/categorias/{id}` actualiza nombre y descripcion con las mismas validaciones; `DELETE /api/v1/categorias/{id}` la anula (204) y no anula los productos clasificados en ella.

- `POST /api/v1/productos/{id}/categorias` crea el vínculo y responde 201; clasificar dos veces en la misma categoría, en una categoría anulada o un producto anulado, devuelve 409; un categoria_id inexistente devuelve 404.
- Al marcar una categoría como principal, la que lo era deja de serlo: el producto nunca queda con dos principales.
- `GET /api/v1/productos/{id}/categorias` lista las categorías del producto indicando cuál es la principal, y `GET /api/v1/productos?categoria_id=` devuelve los productos de una categoría sin duplicar filas.

HU-P2-03: Administrar el catálogo desde la interfaz web

Como administrador de Food Store, **quiero** gestionar los productos desde una pantalla web, **para** no depender de herramientas técnicas para operar el catálogo.

Criterios de aceptación

- Al abrir la aplicación se listan los productos traídos desde la API, cada uno con su precio y su stock con el símbolo de su unidad, y con los indicadores de disponible y hay_stock.
- El botón de alta abre un modal con el formulario vacío y el selector de unidad cargado desde la API; el de edición lo abre con los datos del producto seleccionado.
- Al guardar, el listado refleja el cambio sin recargar la página; el modal se cierra al guardar y al cancelar. Un 409 por nombre repetido se muestra dentro del modal sin cerrarlo.
- Anular un producto lo quita del listado; cambiar su disponibilidad se refleja en la fila correspondiente.
- Los componentes del listado reciben los datos por props tipadas y no consultan la API por su cuenta.
- Mientras se cargan los datos se muestra un indicador; si la API falla se muestra un mensaje de error; si no hay productos se muestra un estado vacío.

8. Criterios de evaluación

Historia de usuario / entregable	Req.	Puntos
Estructura del backend, persistencia con SQLModel, modelo de datos y seed	R1	15
HU-P2-01 — Administrar los productos del catálogo	R2	30
HU-P2-02 — Organizar el catálogo en categorías y clasificar productos	R3	25
HU-P2-03 — Administrar el catálogo desde la interfaz web	R4	30
TOTAL		**100**
Mínimo para aprobar		**60**

9. Proceso de Evaluación-Parcial 2

- 📁 **Fase 1: Entrega del Proyecto** El Parcial 2-desarrollo se entregará entre el sábado 31 de octubre y el miércoles 11 de noviembre de 2026.
- 💻 **Fase 2: Cuestionario en Plataforma** Los estudiantes que cuenten con el Parcial 2-desarrollo aprobado accederán a esta segunda etapa. El sábado 14 de noviembre de 2026, los alumnos rendirán un cuestionario de 20 preguntas a través de la plataforma Moodle, enfocadas exclusivamente en el trabajo entregado. 💡 *Nota:* Los ejemplos de este tipo preguntas ya se encuentran disponibles en el aula virtual.

- 🗣️ **Fase 3: Defensa Oral Sincrónica** Aquellos alumnos que aprueben el cuestionario de Moodle pasarán a la última instancia, la cual consistirá en un encuentro oral sincrónico a través de Zoom. En esta evaluación se realizará una pregunta directa vinculada al Parcial 2-desarrollo y al contenido de las unidades: 4,5 y 6. 📌 *Nota:* El listado completo de las preguntas posibles para esta instancia está publicado en el aula virtual.